

Compiler Design

Janis Hutz
<https://janishutz.com>

September 30, 2026



“A funny quote by a professor”

- Prof. Dr. Professor Name, YEAR

HS2026, ETHZ
Summary of the Script and Lectures

Contents	
1	Introduction 2
2	OCAML 3
3	x86Lite 4
3.1	Introduction 4
3.2	Instructions & Syntax 4
3.3	List of instructions 4
3.3.1	Arithmetic Instructions 4
3.3.2	Logic/Bit Manipulation Instructions 4
3.3.3	Control Flow, Blocks and Labels 5
3.3.4	Condition Flags & Codes, Conditional Instructions 5
3.3.5	Stack Operations & Memory Model 5
3.4	Calling Conventions 5
3.4.1	Callee vs Caller Saved registers 5
3.4.2	Arguments 6
4	LLVM 6
5	Parsing 6
6	Lexing 6
7	Optimizations 6

1 Introduction

This summary is still a very early version and structure and content will still change and errors will be fixed.

It is also likely that it won't be as extensive and detailed as some of my others due to the more applied nature of the course and the fact that a Cheat Sheet can be brought to the exam.

2 OCAML

OCAML is a compiled, functional programming language and a descendant of the Meta Language programming language. In contrast to other functional programming languages (such as Haskell), it is actually used in the wild by more than two people.

File: code/ocaml/00_basics.ml

```

1  (* comments are like this and can be nested *)
2  let a_function (a : int) (b : int) : int = a * b
3
4  (* We can use the following to print ANYTHING upon execution *)
5  let () = Printf.printf "Format String (just like C's printf)"
6
7  (* We can also use ; to sepearate statements. The last statement's value is returned
8   Furthermore, this definition of `a_function` shadows the previous one *)
9  let a_function (a : int) : int =
10     Printf.printf "Print anything here";
11     a * a
12
13  let vars_in_func (a : int) =
14     let x = a_function a in
15     Printf.printf "Value is %d" x
16
17  (* This is a simple example of a generic funciton and pattern matching *)
18  let is_empty (a : 'a list) : bool =
19     match a with
20     | [] -> true
21     | _ :: _ -> false (* The underscores are, just like in Haskell, wildcards *)
22
23  let if_func (v1 : string) (v2 : string) : string =
24     if v1 = v2 then "match" else "no match"
25
26  (* Recursive function *)
27  let rec double_all (arg : int list) : int list =
28     match arg with [] -> [] | a :: al -> (a * a) :: double_all al
29
30  (* Recursive Tail function. This is more efficient than the above because it's compiled
31   to a simple loop. Furthermore, the fold functions behave in this way *)
32  let double_all_t (arg : int list) : int list =
33     let rec aux a acc = match a with [] -> acc | v :: vl -> aux vl (v :: acc) in
34     List.rev (aux arg [])
35
36  let print_list (list : 'a list) = List.iter (Printf.printf "%d") list
37
38  (* We can also use pattern matching for the tuple contents *)
39  let tup = 100, "String", true
40  let second (_, x, _) = x

```

OCAML is dynamically typed (i.e. types don't have to be annotated), but they can be, if you so choose. As with Python and all other languages where type annotation is optional, you are strongly encouraged to annotate types, especially for more complex expressions, as that can massively reduce the likeliness of errors.

The first graded assignment's first 300 ish lines also give you a very neat overview over how OCAML works, so absolutely read through those.

For Printf, the format string is equivalent to that of C, a solid cheat sheet can be found [here](#)

3 x86Lite

3.1 Introduction

x86lite is a subset of the full x86 ISA, with only about 20 instructions and only 64-bit signed integers, nothing else. It is further interoperable with x86.

As a quick reminder, the following registers are relevant for x86lite:

▪ rax	▪ rsi	▪ r08	▪ r12
▪ rbx	▪ rdi	▪ r09	▪ r13
▪ rcx	▪ rbp (Base Pointer)	▪ r10	▪ r14
▪ rdx	▪ rsp (Stack Pointer)	▪ r11	▪ r16

Further, the rip points to the next instruction. Then, the CPU has processor state registers, such as OF (set, if the last instruction caused an overflow), ZF (set if last result was zero), etc.

On x86, the heap grows upwards, the stack grows downwards, where the code and data section that precedes the heap in the address space contains program code, constants and globals.

3.2 Instructions & Syntax

x86lite uses the AT&T syntax, where the source comes *before* the destination, immediates are prefixed with a \$ and registers with a %. Each mnemonic has suffixes q = quadword (4 words), l = long (2 words), w = word (16 bits) and b = byte (8 bit), for example movq \$5, %rax. This syntax is prevalent in the UNIX ecosystem (thus is *objectively superior*...)

The following operands can be passed to instructions:

- **Immediate** (Imm):
- **Label** (Lbl):
- **Register** (Reg):
- **Machine Address** (Ind): Using the format disp(base, index, scale), with the address computed as disp + base + index * scale, with base and index registers (index cannot be the rsp reg) and disp and scale int32 immediates, where in x86lite scale is by default 8

3.3 List of instructions

A list of instructions is also provided on the Moodle page for the course.

3.3.1 Arithmetic Instructions

Instruction	Explanation	Example
negq DEST	2's complement negation	negq %rax
addq SRC, DEST	$DEST \leftarrow DEST + SRC$	addq %rbx, %rax
subq SRC, DEST	$DEST \leftarrow DEST - SRC$	subq \$4, %rsp
imulq SRC, DEST	$DEST \leftarrow DEST \times SRC$	imulq \$4, %rax

3.3.2 Logic/Bit Manipulation Instructions

Instruction	Explanation	Example
notq DEST	bitwise not	notq %rax
andq SRC, DEST	$DEST \leftarrow DEST \& SRC$	andq %rbx, %rax
orq SRC, DEST	$DEST \leftarrow DEST SRC$	orq \$4, %rsp
xorq SRC, DEST	$DEST \leftarrow DEST \oplus SRC$	xorq \$2, %rax
sarq Amt, DEST	$DEST \leftarrow DEST \gg Amt$ (Arithmetic right shift)	sarq \$4, %rax
shrq Amt, DEST	$DEST \leftarrow DEST \ggg Amt$ (Logical right shift)	shrq \$1, %rsp
shlq Amt, DEST	$DEST \leftarrow DEST \lll Amt$ (Logical left shift)	shlq %rbx, %rax

3.3.3 Control Flow, Blocks and Labels

As you are probably aware of, x86 assembly organizes code into *labeled blocks*. Labels are translated away by the linker and loader and the code begins executing at a designated code label (usually “main”).

To call a subroutine at a given label, we can use the `call LABEL` instruction and we can use the `ret` instruction to return from the procedure.

Instruction	Description	Notes
<code>jmp SRC</code>	$\text{rip} \leftarrow \text{SRC}$	Jump to location in SRC
<code>call SRC</code>	Push <code>rip</code> ; $\text{rip} \leftarrow \text{SRC}$	Push program counter (<code>rip</code>) onto stack, decrement <code>rsp</code>
<code>ret</code>	Pop into <code>rip</code>	Pop top of stack into <code>rip</code> , increment <code>rsp</code>

This means that the `call` and `ret` instructions act like some sort of abstraction of `jmp`.

3.3.4 Condition Flags & Codes, Conditional Instructions

The following flags are set as side effects from normal instructions:

- *OF overflow*: is set when the result is too big or small to fit in the 64 bit register
- *SF sign*: set to the sign of the result
- *ZF zero*: set when the result is 0

From these flags, we can define *condition codes*:

Condition Codes	Condition	Description
<code>e</code>	<code>ZF</code>	Equal / Zero
<code>ne</code>	<code>~ZF</code>	Not Equal / Not Zero
<code>g</code>	<code>~(SF^OF)&~ZF</code>	Greater (signed)
<code>ge</code>	<code>~(SF^OF)</code>	Greater or equal (signed)
<code>l</code>	<code>SF^OF</code>	Less (signed)
<code>le</code>	<code>(SF^OF) ZF</code>	Less or equal (signed)

Instead of manually computing `SRC1 - SRC2` manually, we can instead use the following instructions to set the condition codes:

Instruction	Description
<code>cmpq SRC2, SRC1</code>	Computes <code>SRC1 - SRC2</code> sets condition flags
<code>setb CC, DEST</code>	<code>DEST's lower byte</code> $\leftarrow$ if <code>CC</code> then 1 else 0
<code>jCC SRC</code>	<code>rip</code> $\leftarrow$ if <code>CC</code> then <code>SRC</code> else fall through

3.3.5 Stack Operations & Memory Model

To load a pointer into a register, we can use `leaq Ind, DEST`, which does `DEST` $\leftarrow$ `addr(Ind)`.

As mentioned previously, the x86 stack grows downwards, so the `pushq SRC` instruction computes `rsp` $\leftarrow$ `rsp` - 8; `Mem[rsp]` $\leftarrow$ `SRC`, where `popq DEST` computes `DEST` $\leftarrow$ `Mem[rsp]`; `rsp` $\leftarrow$ `rsp` + 8.

3.4 Calling Conventions

As covered in SPCA, the stack in C stores local variables and other book-keeping data. Global variables are stored in the code and data section at the low end of the address space. Further, remember that the `rbp` (base pointer) register by convention contains the *previous value* of `rsp`.

3.4.1 Callee vs Caller Saved registers

By convention, the registers `rbp`, `rsp`, `rbx`, `r12`, `r13`, `r14`, `r15` are *Callee-Saved*, i.e. it is the duty of the *subroutine* (the callee) to restore them. All other registers are *Caller-Saved*, meaning that the callee can freely use them.

3.4.2 Arguments

Function arguments one through six are stored in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, respectively, any extra arguments are to be stored on the stack in right-to-left order, meaning that for $n > 6$, the n -th argument is at $((n - 7) + 2) * 8 + \text{rbp}$.

The return value is stored in `rax` and the stack must be 16-byte aligned on a call. Further, there is a 128 byte “red zone”, which is a scratch pad for the callee’s data, which is there for optimization.

4 LLVM

5 Parsing

6 Lexing

7 Optimizations